

软件过程定义：辞典编撰辅助平台开发

总体概括

本项目的软件过程可以概括为：在复用成熟 Graph-RAG 与多智能体框架的基础上，以“审、比、勘、改”训诂思维链为核心，围绕数据准备、Agent 搭建、前后端集成和专家闭环验证进行小团队敏捷迭代开发。项目不追求从零实现全部底层框架，而是把主要工程资源投入到古汉语辞典编撰场景中的流程设计、证据组织、推理链构建和平台化落地。

项目背景简介

本项目为基于训诂学的汉字释义知识图谱构建与辞典编撰辅助平台开发，旨在为国家“十四五”重大工程《汉语大字典》第三版修订提供智能化的辅助工具。

传统辞典编撰面临语料多源异构和人工考证繁琐、周期长等痛点。本项目通过构建面向辞典编撰专家和古汉语研究者的“智能考据专家系统”，打通古汉语知识链路。系统底层采用 Neo4j 与 MySQL 搭建数据基座，中层利用 Qwen 3 - 8B 大模型配合 Graph-RAG（基于图的检索增强）以及多智能体（Multi-Agent）进行复杂语义推理与循证辩论，上层通过 React 和 FastAPI 实现专家可视化交互与溯源服务。系统核心实现了“审、比、勘、改”的训诂思维链（CoT），大幅提升传统典籍修订的效率与严谨性。

软件过程定义

针对包含大模型推理、知识图谱构建以及高度依赖领域专家反馈的特性，本项目的软件过程定义为一个融合了领域数据工程的敏捷迭代生命周期过程。具体划分为以下五个核心过程阶段：

1. 领域需求与数据准备过程（定义与采集）

- 业务建模与需求分析：** 与训诂学专家对接，明确传统编撰的体例标准与隐性规则。
- 语料标准化与清洗：** 对多源异构的字典进行结构化处理，提取“字头-义项-书证”三元组。

2. 核心算法与引擎研发过程（研究与原型）

- 知识发现与图谱构建：** 应用“义素分析法”将自然语言释义解构为离散特征向量，构建底层引申链图谱。
- 智能体与检索增强设计：** 基于现有 Graph-RAG 与多智能体框架进行场景化适配，配置宏微观双层检索策略，并搭建保守派、革新派与裁判 Agent 的辩论流程。

3. 系统架构与增量开发过程（构建与集成）

- 前后端分离开发：** 敏捷迭代开发 React 前端界面与 Node.js/FastAPI 后端服务。

- **模型与系统集成：**将 Qwen 模型推理引擎与业务系统对接，实现上下层通信交互。

4. 人机协同与闭环验证过程（测试与评审）

- **自动化辅助测试：**校验各模块 API 接口及图数据库查询性能。
- **专家闭环反馈（UAT）：**由编撰专家对多智能体的辩论结果和生成的“待勘问题清单”进行终局初裁和质量把关。

5. 规范定稿与部署发布过程（交付与维护）

- **成果导出与知识重构：**输出排版规范、附带修改日志的修订定稿条目，动态修剪底层图谱。
- **部署与持续监控：**系统上线运行，监控大模型幻觉率与系统响应延迟。

定义的来源

本软件过程的定义并不是对某一个标准的机械照搬，而是以课程中介绍的软件过程标准为主干，再结合本项目“数据驱动、模型推理、专家协同、平台交付”的实际特点进行综合形成。其来源主要包括以下几个方面：

1. ISO/IEC 12207（软件生命周期过程）

- **来源作用：**ISO/IEC 12207 为软件从需求、设计、实现、集成、测试、交付到维护提供了完整的生命周期框架，是本项目划分过程阶段的主要依据。
- **在本项目中的体现：**项目中的“领域需求与数据准备过程”对应需求分析和业务建模；“核心算法与引擎研发过程”对应软件设计与实现；“系统架构与增量开发过程”对应软件集成；“人机协同与闭环验证过程”对应软件验证、确认和评审；“规范定稿与部署发布过程”对应交付、运行和维护。
- **具体借鉴点：**除了主要生命周期过程外，本项目还借鉴了其支持过程思想，例如配置管理、质量保证、评审管理和文档管理。对于知识图谱结构、提示词版本、Agent 配置、接口文档和专家反馈记录，都需要进行版本留存，避免模型输出和人工修订过程不可追踪。

2. ISO/IEC 15504 与 ISO/IEC 33001（过程评估与能力成熟度）

- **来源作用：**ISO/IEC 15504 和 ISO/IEC 33001 强调过程能力、过程评估和持续改进，为本项目提供了“如何判断过程是否有效”的思路。
- **在本项目中的体现：**本项目不是一次性完成后即结束，而是需要在多轮辞典条目、书证数据和专家反馈中不断优化。因此，过程定义中加入了迭代评估机制，例如语料清洗准确率、义项对齐一致性、Graph-RAG 检索命中率、多智能体辩论结果可解释性、专家采纳率等指标。
- **具体借鉴点：**项目希望逐步从“可执行”提升到“可重复、可度量、可优化”。早期阶段先保证平台能够跑通完整流程，中期重点固化稳定的提示词模板、Agent 角色配置和接口规范，后期再根据专家反馈持续改进推理链质量。

3. CRISP-DM 与数据工程实践

- **来源作用：**辞典编撰辅助平台具有明显的数据密集型特征，单纯按照传统软件开发过程不足以覆盖多源语料理解、清洗、建模、评估等工作。因此，过程定义吸收了 CRISP-DM 中“业务理

解、数据理解、数据准备、建模、评估、部署”的思路。

- **在本项目中的体现：**项目前期不仅要写功能代码，还要理解《汉语大字典》《辞源》等辞书资料的体例差异，对“字头-义项-书证”进行结构化抽取，并将文本数据转换为图数据库和检索索引能够使用的形式。
- **具体借鉴点：**数据准备不被视为编码前的附属工作，而被定义为软件过程中的核心活动。因为 Graph-RAG 的效果、多智能体辩论的质量以及最终修订建议的可靠性，都取决于数据源是否规范、证据链是否完整、引文是否可追溯。

4. 敏捷开发与原型迭代实践

- **来源作用：**本项目涉及大模型、知识图谱和训诂学规则，技术不确定性和需求探索性较强，适合采用敏捷开发中的短周期迭代和持续反馈方式。
- **在本项目中的体现：**每一轮迭代可以围绕一个小目标展开，例如先完成一个字头的义项抽取，再完成一组义项的横向比对，再完成“审比勘改”链路中的某一个 Agent，最后接入前端页面进行展示和专家反馈。
- **具体借鉴点：**团队不在一开始追求庞大而完整的系统，而是先做可运行原型，再逐步增强数据覆盖范围、推理规则、界面交互和结果导出能力。

5. 项目自身技术路线与领域专家工作流程

- **来源作用：**软件过程还来源于辞典编撰工作的真实业务流程，即专家在修订时通常需要先审读单本辞书，再比较多源释义，之后结合文献证据进行考订，最后形成规范化修改意见。
- **在本项目中的体现：**因此，本项目将领域工作流程抽象为“审、比、勘、改”四步训诂思维链：先做单源深度解析，再做多源横向对齐，然后通过循证辩论形成待勘意见，最后输出可追溯的修订建议。
- **具体借鉴点：**这一来源决定了本项目的软件过程不能只围绕“功能是否实现”展开，还必须围绕“证据是否充分、推理是否清晰、专家是否能够复核”展开。

关于软件过程剪裁

作为一个融合了人文社科（训诂学）与前沿计算机科学（大模型、图谱）的跨学科研发项目，完全照搬重量级的传统软件过程标准是不合适的。因此，对标准过程进行了以下针对性**剪裁（Tailoring）**：

1. 复用成熟框架，减少底层重复开发：

- **剪裁想法：**多智能体框架和 Graph-RAG 检索增强框架不作为本项目从零研发的重点，而是尽量复用已有成熟框架和现成工程组件。项目主要根据辞典编撰场景进行适配，例如定义 Agent 角色、编排调用顺序、配置检索策略、组织证据上下文和沉淀提示词模板。这样可以减少底层框架开发成本，把有限时间投入到真正具有项目特色的训诂推理流程上。

2. 弱化重量级文档，强化“审比勘改”思维链构建：

- **剪裁想法：**传统瀑布模型中大量前置规格说明书，对于大模型提示词工程和多智能体协同的指导作用有限。本项目保留必要的需求说明、接口说明和数据规范，但不把主要精力放在编写厚

重文档上，而是放在“审、比、勘、改”四步思维链的可运行实现上。

- **重点工作：** 需要明确每一步的输入、输出和判断依据。例如“审”阶段输出单本辞书的义素拆解和引申关系，“比”阶段输出多源释义合并建议，“勘”阶段输出待勘问题清单和证据链，“改”阶段输出规范化修订建议与修改日志。

3. 将 Agent 搭建作为独立核心任务：

- **剪裁想法：** 在传统软件过程中，模块划分往往以功能页面或后端服务为单位。但本项目的关键模块是多个具有不同立场和任务的 Agent，因此需要把 Agent 设计、提示词模板、工具调用权限、记忆与日志记录单独作为过程重点。
- **重点工作：** 多智能体部分可复用已有编排框架，但保守派 Agent、革新派 Agent、裁判 Agent 等角色的职责边界、证据使用规则和输出格式需要结合训诂学场景重新设计。过程管理中要记录每个 Agent 的版本、提示词变化和典型输出案例，便于后续复盘。

4. 突出前后端链接与系统集成，压缩非必要的独立模块研发：

- **剪裁想法：** 项目最终要服务于辞典编撰专家，因此不能停留在算法脚本或命令行原型。相比单独追求算法模块复杂度，本项目更需要保证前端页面、后端接口、图数据库、模型推理服务和专家反馈入口能够顺畅打通。
- **重点工作：** 前端负责展示字头、义项、书证、检索证据、Agent 辩论过程和修订建议；后端负责封装数据查询、Graph-RAG 检索、Agent 调用和结果存储；接口层需要保证每一步思维链结果都能被前端查看、追溯和人工修改。

5. 采用四人左右小团队分工协作：

- **剪裁想法：** 由于团队规模较小，不适合设置过多层级化角色，也不需要完整套用大型项目中的项目经理、配置管理员、测试经理等专职岗位。更合理的方式是采用一人多职、模块负责制和短周期同步机制。
- **建议分工：** 第一名成员负责语料整理、知识图谱建模和 Graph-RAG 数据接入；第二名成员负责“审比勘改”思维链设计、提示词模板和 Agent 角色搭建；第三名成员负责后端服务、模型调用、数据库接口和前后端 API；第四名成员负责 React 前端页面、交互展示、专家反馈入口和系统联调测试。
- **协作方式：** 每轮迭代由四人共同确定一个可验收目标，例如完成一个字头的全流程演示。随后分别推进数据、Agent、后端和前端工作，最后通过集成测试和专家反馈统一验收。

6. 定制化测试与专家闭环验证：

- **剪裁想法：** 古汉语释义的准确性缺乏绝对的自动化“测试预言 (Test Oracle)”，因此不能只依赖单元测试和接口测试。本项目保留必要的自动化测试，用于验证接口、数据库查询和页面功能；同时将专家评审作为核心确认手段。
- **重点工作：** 对每次输出的修订建议，都需要保留来源证据、Graph-RAG 检索结果、Agent 辩论过程和专家修改意见。这样既能保证系统结果可复核，也能让后续迭代知道应当优化数据、提示词还是 Agent 编排。